

PEARL: A Partial Evaluation Toolbox for a Reversible Language

Louis Marott Normann

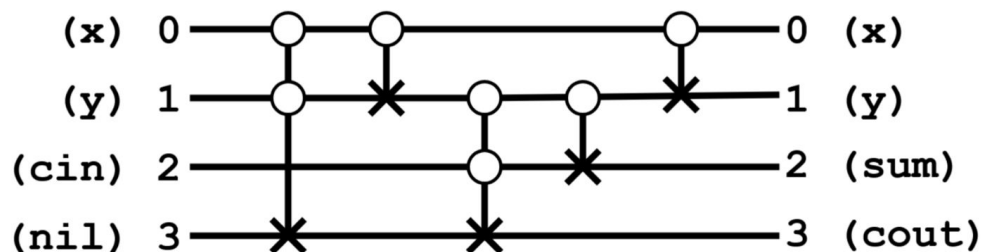
Robert Glück

10/07/2026

DIKU, University of Copenhagen, Denmark



A Teaser



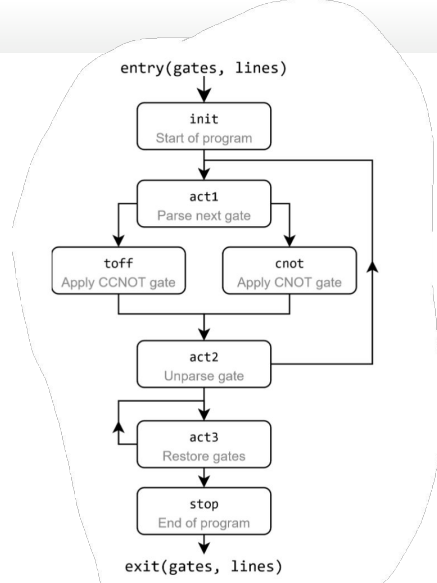
[CCNOT 0 1 3, CNOT 0 1, CCNOT 1 2 3, CNOT 1 2, CNOT 0 1]

```
(lines) -> (lines) with (v1 v2 v3)
init: entry
  v1 <- lines['0']
  v2 <- lines['1']
  v3 <- lines['3']
  v3 ^= v1 && v2
  lines['3] <- v3
  lines['1] <- v2
  lines['0] <- v1
  v1 <- lines['1]
  v2 <- lines['2]
  v2 ^= v1
  lines['2] <- v2
  exit
  v1 <- lines['0']
  v2 <- lines['1']
  v2 ^= v1
  lines['1] <- v2
  lines['0] <- v1
```

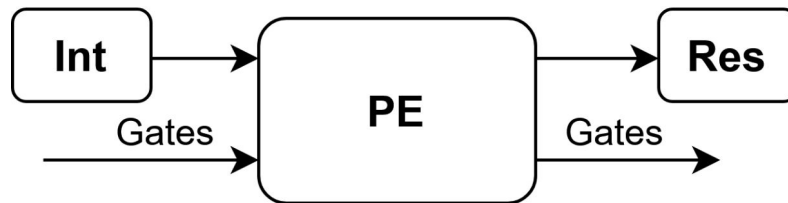
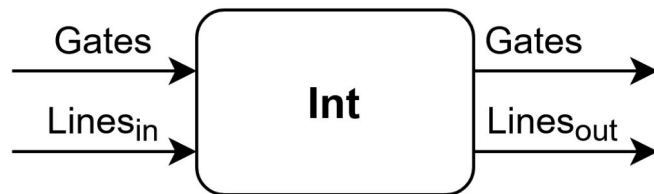
Using only an interpreter?!

```
(gates lines) -> (gates lines)
with (gate gatesRev ls l1 l2 l3 v1 v2 v3)
init: entry
      goto act1
act1: fi gatesRev from act2 else init
      ((gate . ls) . gates) <- gates
      if gate = 'CCNOT goto toff else cnot
toff: from act1
      [l1, l2, l3] <- ls
      v1 <- lines[l1]
      v2 <- lines[l2]
      v3 <- lines[l3]
      v3 ^= v1 && v2
      lines[l3] <- v3
      lines[l2] <- v2
      lines[l1] <- v1
      ls <- [l1, l2, l3]
      goto act2
```

```
cnot: from act1
      [l1, l2] <- ls
      v1 <- lines[l1]
      v2 <- lines[l2]
      v2 ^= v1
      lines[l2] <- v2
      lines[l1] <- v1
      ls <- [l1, l2]
      goto act2
act2: fi gate = 'CCNOT from toff else cnot
      gatesRev <- ((gate . ls) . gatesRev)
      if gates goto act1 else act3
act3: fi gates from act3 else act2
      (gate . gatesRev) <- gatesRev
      gates <- (gate . gates)
      if gatesRev goto act3 else stop
stop: from act3
      exit
```



The Trick



A Reversible Language

Three Injective Steps

Reversible Assignment:

$$x \oplus = e$$

- **$x \ += \ 3$:**
Add 3 to **x** .
- **$x \ ^= \ 3$:**
Initialize **x** to 3.
- **$x \ -= \ x$:**
 x must not occur on the RHS.

Reversible Replacement:

$$q \leftarrow q$$

- **$xs \leftarrow (x.xs)$:**
Push **x** on **xs** .
- **$(x.xs) \leftarrow xs$:**
Pop from **xs** to **x** .
- **$(x.x) \leftarrow (y.y)$:**
Patterns are linear.

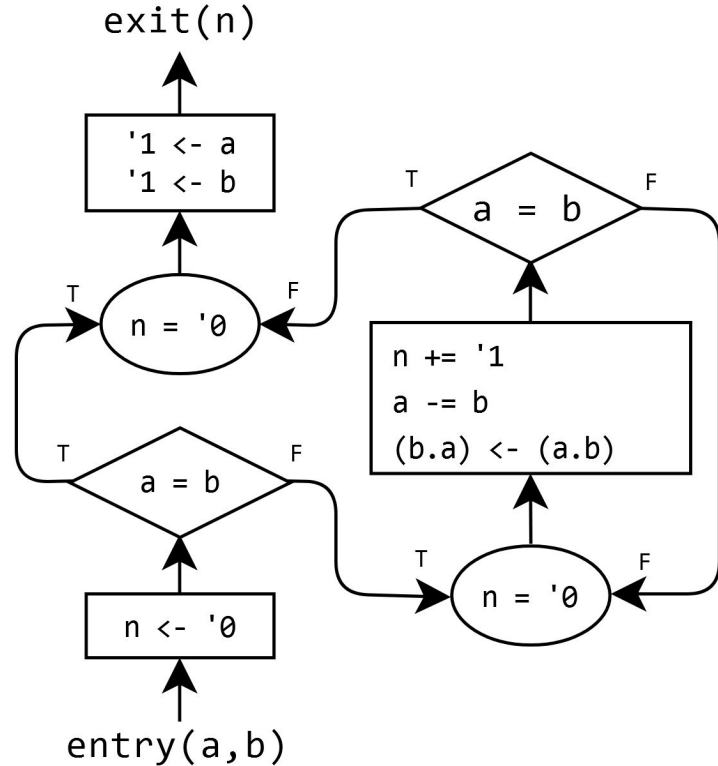
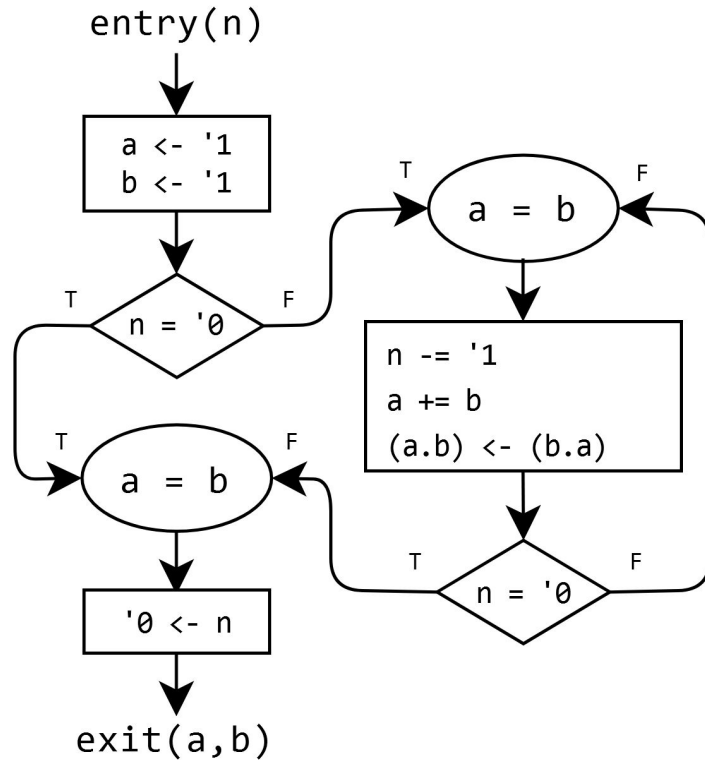
Assertions:

$$\text{assert}(e)$$

- **$\text{assert}(x)$:**
 x must be true.
- **$\text{assert}(!x)$:**
 x must be false.
- **$\text{assert}(\text{'nil'})$:**
Run-time error.

Inversion of ARL

An Invertible Program



Partial Evaluation of ARL

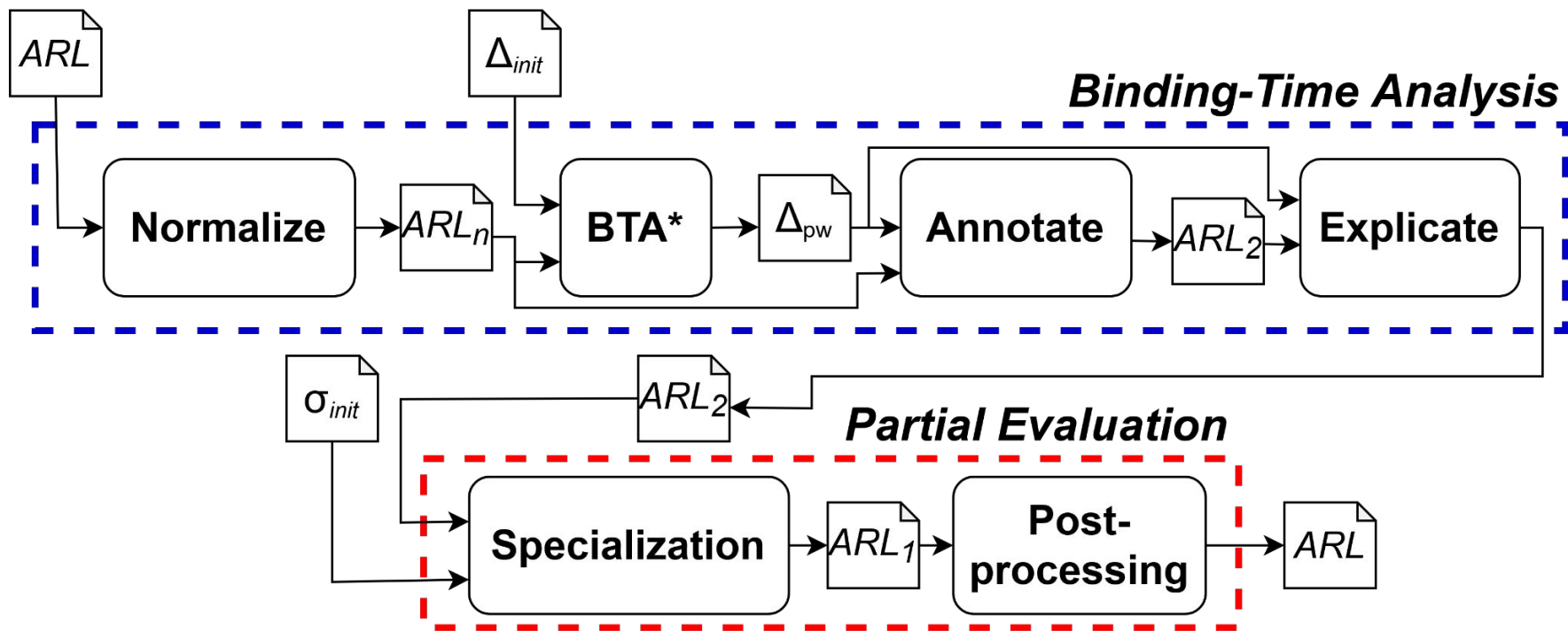
A Quick Recap

The offline approach [Jones et al., 1993, Hatcliff, 1999]:

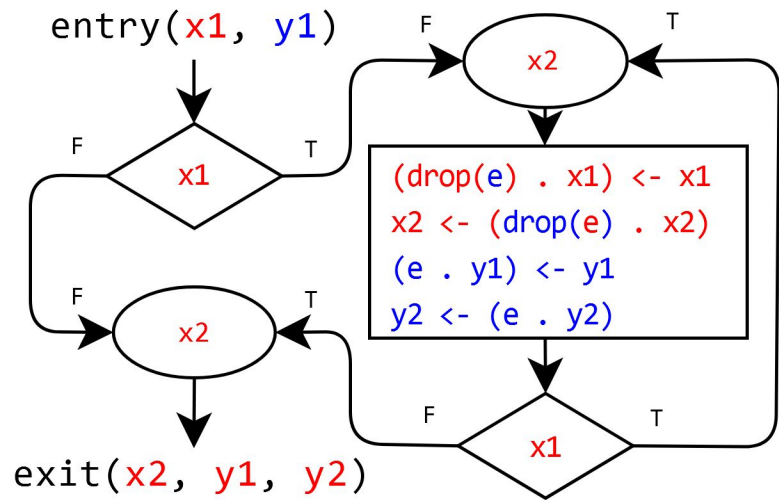
- Binding-time analysis (Uniform)
- Specialization (Polyvariant)

Key issues for reversible programs:

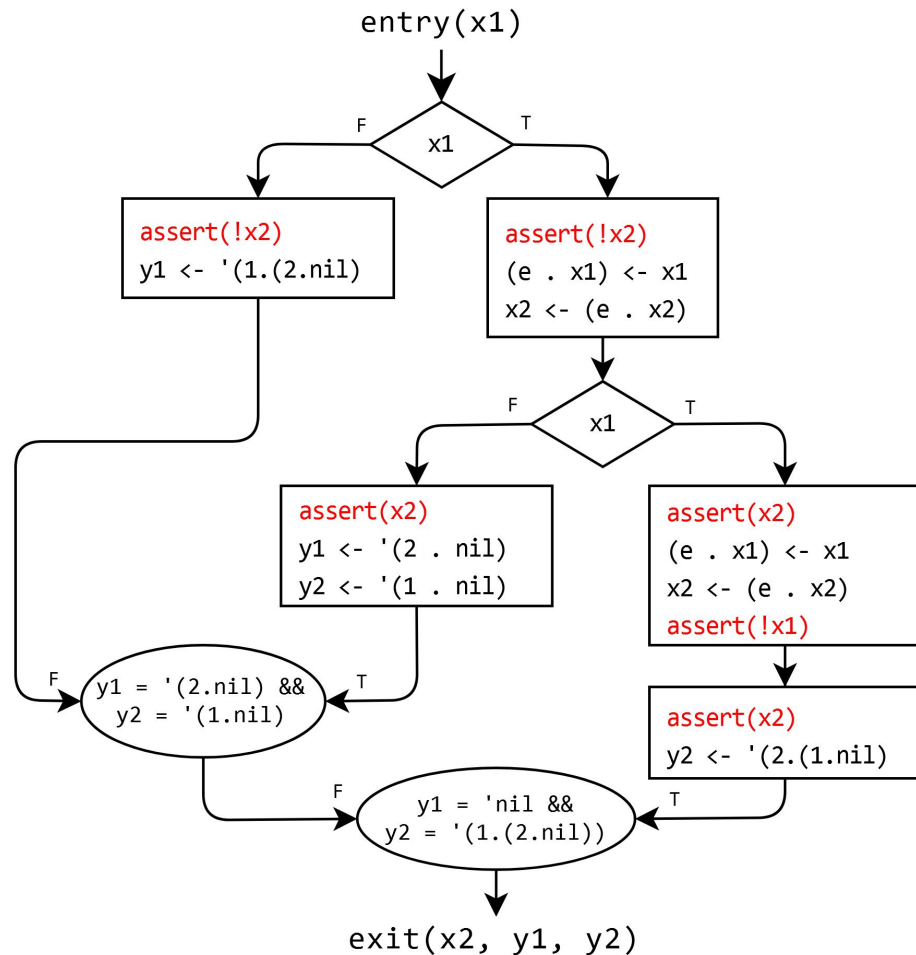
- Multiple exits
- Malformed control-flow statements
- Static store collision



Full Example



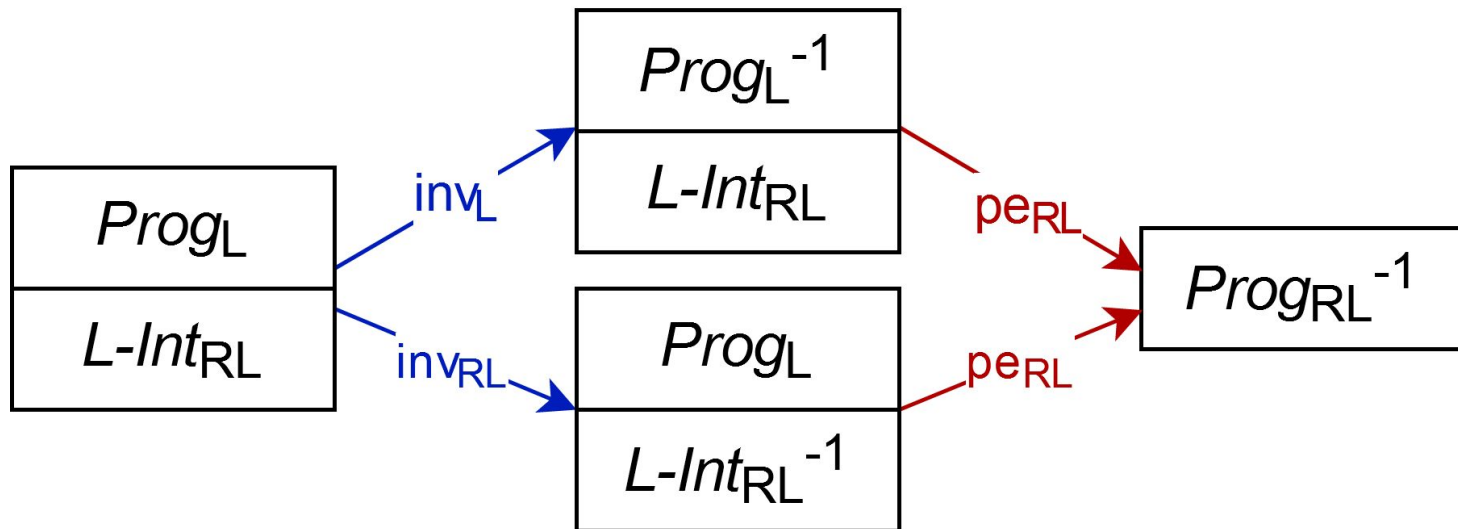
Specialized w.r.t. $y1 \rightarrow (1, (2, \text{nil}))$



Experiments

Inversion Projection

(i) Traditional Inversion & 1. Futamura Projection



(ii) Source Inversion & 1. Inversion Projection

Fin

<https://github.com/Yakokse/PEARL>





Partial Evaluation Appendix

Benchmarks, Size

	<i>Source Program</i>		<i>Uniform BTA</i>			<i>Pointwise BTA</i>		
Program	Blocks	LOC	Blocks	LOC	Elim.	Blocks	LOC	Elim.
<i>Partition</i>	3	10	1	(12) 8	4/4	1	(12) 8	4/4
<i>Repeat</i>	3	11	1	(13) 9	4/4	1	(13) 9	4/4
<i>Encryption</i>	6	23	58	232	0/0	58	232	0/0
<i>PRNG</i>	7	26	7	26	0/0	45	(194) 166	28/28
<i>DoubleRev, 1</i>	3	10	3	10	0/0	10	(40) 32	8/8
<i>DoubleRev, 2</i>	3	10	3	10	0/0	31	(124) 102	22/22
<i>RLD, Counts</i>	7	21	(9) 1	(52) 22	14/17	(9) 1	(52) 22	14/17
<i>RLD, Chars</i>	7	21	7	21	0/0	(17) 12	(67) 52	5/5
<i>2RM, Double</i>	20	52	3	12	0/0	3	12	0/0
<i>2RM, 1 to N</i>	20	52	3	12	0/0	3	12	0/0
<i>RTM, Cons</i>	24	70	(7) 4	(29) 23	0/5	(7) 4	(29) 23	0/5
<i>RTM, Inc</i>	24	70	22	74	0/6	22	74	0/6
<i>RTM, Flip</i>	24	70	25	82	0/6	25	82	0/6

Benchmarks, Speed

	<i>Source Program</i>			<i>Uniform BTA</i>				<i>Pointwise BTA</i>			
Program	S#	J#	Time	S#	J#	Time	Speed	S#	J#	Time	Speed
<i>Partition</i>	10	4	18	(10) 6	0	(10) 6	3.00x	(10) 6	0	(10) 6	3.00x
<i>Repeat</i>	11	4	18	(11) 7	0	(11) 7	2.57x	(11) 7	0	(11) 7	2.57x
<i>Encryption</i>	88	58	204	67	38	143	1.43x	67	38	143	1.43x
<i>PRNG</i>	282	141	564	282	141	564	1.00x	(59) 45	19	(97) 83	6.80x
<i>DoubleRev, 1</i>	12	4	20	12	4	20	1.00x	(12) 7	5	(22) 17	1.18x
<i>DoubleRev, 2</i>	40	11	62	40	11	62	1.00x	(33) 21	13	(59) 47	1.32x
<i>RLD, Counts</i>	25	19	63	(33) 20	(7) 0	(47) 20	3.15x	(33) 20	(7) 0	(47) 20	3.15x
<i>RLD, Chars</i>	25	19	63	25	19	63	1.00x	(30) 24	(14) 13	(58) 50	1.26x
<i>2RM, Double</i>	202	321	844	42	11	64	13.19x	42	11	64	13.19x
<i>2RM, 1 to N</i>	202	321	844	42	11	64	13.19x	42	11	64	13.19x
<i>RTM, Cons</i>	28	25	78	(13) 14	(4) 2	(21) 18	4.33x	(13) 14	(4) 2	(21) 18	4.33x
<i>RTM, Inc</i>	216	250	716	41	41	123	5.82x	41	41	123	5.82x
<i>RTM, Flip</i>	324	381	1086	51	59	169	6.43x	51	59	169	6.43x